

CFGS Desenvolupament d'Aplicacions Multiplataforma

CFGS Desenvolupament d'Aplicacions Web

Bases de dades

Jordi Quesada
jordi.quesada@iticbcn.cat

Llicència.



Atribución-NoComercial-CompartirIgual
4.0 Internacional (CC BY-NC-SA 4.0)

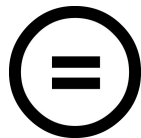
El contingut està disponible sota la llicència
[Creative Commons Reconeixement-NoComercial-CompartirIgual](https://creativecommons.org/licenses/by-nc-sa/4.0/)



Reconeixement — Heu de reconèixer l'autoria de manera apropiada, proporcionar un enllaç a la llicència i indicar si heu fet algun canvi. Podeu fer-ho de qualsevol manera raonable, però no d'una manera que suggereixi que el llicenciador us dóna suport o patrocina l'ús que en feu.



NoComercial — No podeu utilitzar el material per a finalitats comercials.



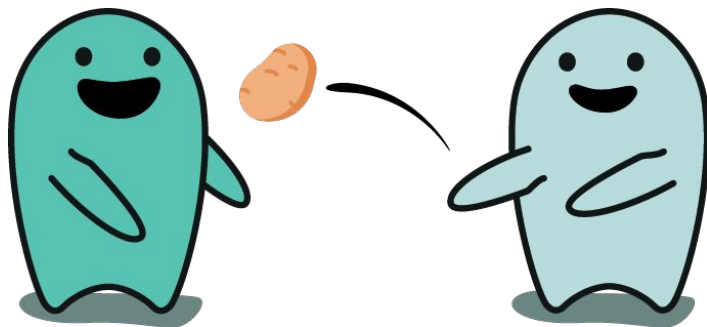
CompartirIgual — Si remescleu, transformeu o creeu a partir del material, heu de difondre les vostres creacions amb la mateixa llicència que l'obra original.

Aspectes avançats

Excepcions

Excepcions

- Una excepció és un error en temps d'execució (divisió per 0, disc dur, memòria...).
- Quan es produeix una excepció, si no es tracta, el control passa al nivell inferior de la pila de crides fins que és tractada.



Excepcions

- Les gestionarem a la zona EXCEPTION
- Tenim 3 tipus d'excepcions
 - Predefinides del sistema
 - Predefinides sense nom
 - De l'usuari

Excepciones

- Les excepciones predefinidas más habituales són:



ZERO_DIVIDE



VALUE_ERROR



TOO_MANY_ROWS



NO_DATA_FOUND



DUP_VAL_ON_INDEX



OTHERS

Excepcions

- Les excepcions predefinides però sense nom més habituals són:
 - Les referents a restriccions check: Codi d'error -2290
 - Les referents a foreign keys: Codi d'error -2291
 - Les referents a not null: Codi d'error -1400
- Podem fer servir SQL_CODE i SQLERRM en el tractament

Excepcions

- Per treballar amb elles caldrà declarar una variable i associar-la

```
DECLARE
  e EXCEPTION;
  pragma exception_init(e, -2292);
BEGIN
  ...
EXCEPTION
  WHEN e THEN
    ...
END;
```

Excepcions

- Finalment tenim les excepcions d'usuari amb les que el programador pot provocar les seves pròpies excepcions
 - `raise`
 - `raise_application_error`

```
BEGIN
```

```
...
```

```
raise_application_error(-20001,'Error');
```

```
...
```

```
END;
```

```
DECLARE
```

```
  e EXCEPTION;
```

```
BEGIN
```

```
...
```

```
raise e;
```

```
...
```

```
END;
```

Cursors

Cursors

- Què passa si un SELECT retorna molts resultats? La instrucció SELECT INTO provoca una excepció TOO_MANY_ROWS i el procediment o funció finalitzen.
- La solució és crear un CURSOR: Una estructura de memòria que permetrà processar un conjunt de resultats fila a fila

Cursors

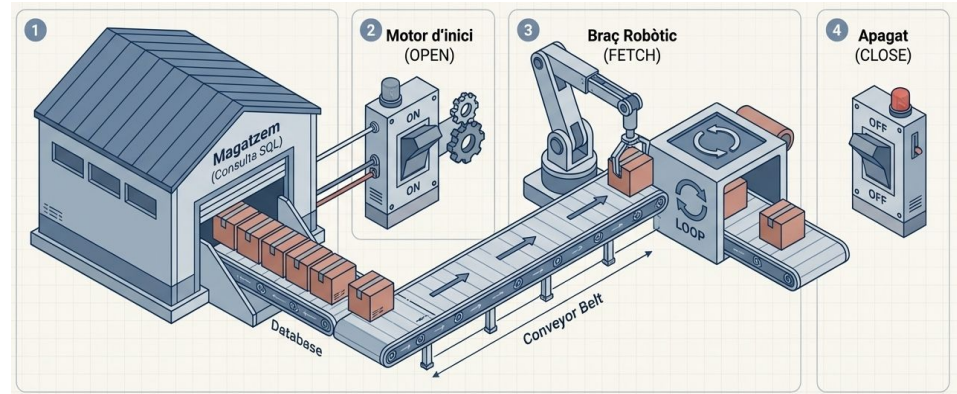
- Cal seguir 4 operacions bàsiques:

Declarar `CURSOR nom IS consultaSQL`

Obrir `OPEN nom;`

Tractar Bucle amb `FETCH`

Tancar `CLOSE nom;`



Cursors

- La comanda FETCH retorna 1 de les files resultat i, a més activa les següents propietats:

%NOTFOUND

Booleà — `true` quan no hi ha més resultats.

%FOUND

Booleà — `true` mentre hi ha resultats.

%ROWCOUNT

Nombre de `FETCH` executats fins al moment.

Cursors

- Exemple:

```
create or replace procedure llista_persones is
    cursor c_persona IS
        select * from persona order by sou;
    x c_persona%ROWTYPE;
BEGIN
    open c_persona;
    dbms_output.put_line('NOM          SOU');
    LOOP
        fetch c_persona into x;
        exit when c_persona%NOTFOUND;
        dbms_output.put_line(x.nom || chr(9) || x.sou);
    END LOOP;
    close c_empl;
end;
```

Cursors

- Amb el bucle for ens podem estalviar alguns passos:

```
create or replace procedure llista_persones is
  cursor c_persona IS
    select * from persona order by sou;
BEGIN
  dbms_output.put_line('NOM   COGNOM   SOU');
  for x in c_persona LOOP
    dbms_output.put_line(x.nom || chr(9) || x.sou);
  END LOOP;
end;
```


Triggers

Triggers

- Un **trigger** és codi PL/SQL que s'executa **automàticament** quan es produeix una acció determinada.
- No rep paràmetres.
- Usos habituals:
 - corregir operacions (per exemple: No fer borrats)
 - evitar accions no permeses (per exemple: no es pot decrementar el sou)
 - monitoritzar usuaris (portar un log de quí fa què).

Triggers

- Caldrà indicar:
 - **timing:** BEFORE, AFTER o INSTEAD OF
 - **event:** INSERT, DELETE, UPDATE
 - **Objecte:** taula o objecte del sistema

```
CREATE OR REPLACE TRIGGER nom
```

```
  timing Event [OR event2...]
```

```
  ON Objecte
```

```
  IS ...
```

Triggers

- A més, podem diferenciar entre:

Trigger d'Acció

S'executa **una vegada**
per acció,
independentment de les
files afectades.

Trigger de Fila

S'executa **una vegada**
per fila afectada.
S'indica amb `FOR EACH`
`ROW`.

Triggers

- Dins del codi tindrem accés a:

:NEW i :OLD

Registres especials als triggers de fila per accedir als valors abans i després de l'operació.

INSERTING, UPDATING, DELETING

Indiquen la operació que ha provocat el trigger

Triggers

- Triggers per evitar accions:

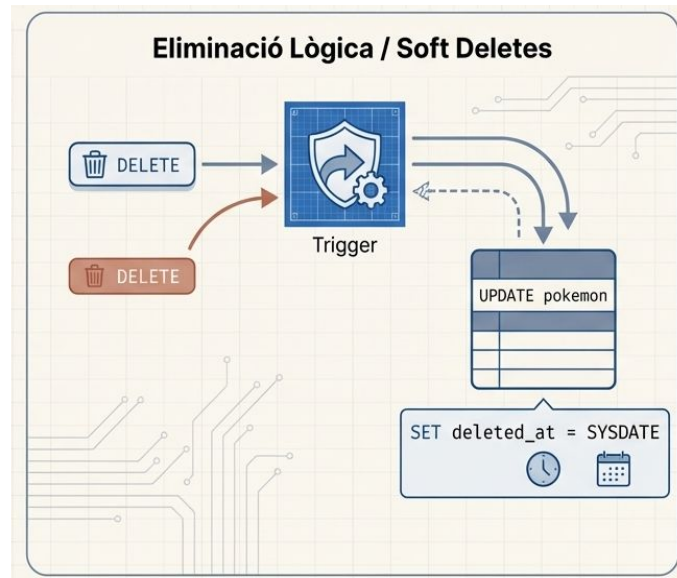


```
create or replace trigger tr_protegeix_sou
before update on empleats for each row
begin
  if :new.sou < :old.sou then
    raise_application_error(-20001, 'Error: no es pot reduir el sou');
  end if;
end;
```

Aquest guardià actua BEFORE (abans) que la dada s'escrigui al disc, garantint que la regla de negoci mai es trenqui.

Triggers

- Triggers per corregir accions:
 - Les dades en realitat normalment no s'eliminen de la base de dades
 - En el seu lloc s'afegeix una marca de borrat i s'afegeix la data d'eliminació
 - És a dir, canviem la operació delete per un update



Triggers

- Per fer-ho farem servir el disparador INSTEAD OF
- Només funciona sobre vistes

```
CREATE view vista_empleats AS (  
    Select * from empleats  
);
```

```
CREATE OR REPLACE TRIGGER trg_soft_delete_empleats  
INSTEAD OF DELETE ON vista_empleats  
FOR EACH ROW  
BEGIN  
    UPDATE empleats  
        SET esborrat = 'S',  
            data_eliminacio = SYSDATE  
        WHERE id = :OLD.id;  
END;
```


Triggers

- O l'altra opció és fer una transacció autònoma per evitar l'error de taules mutants

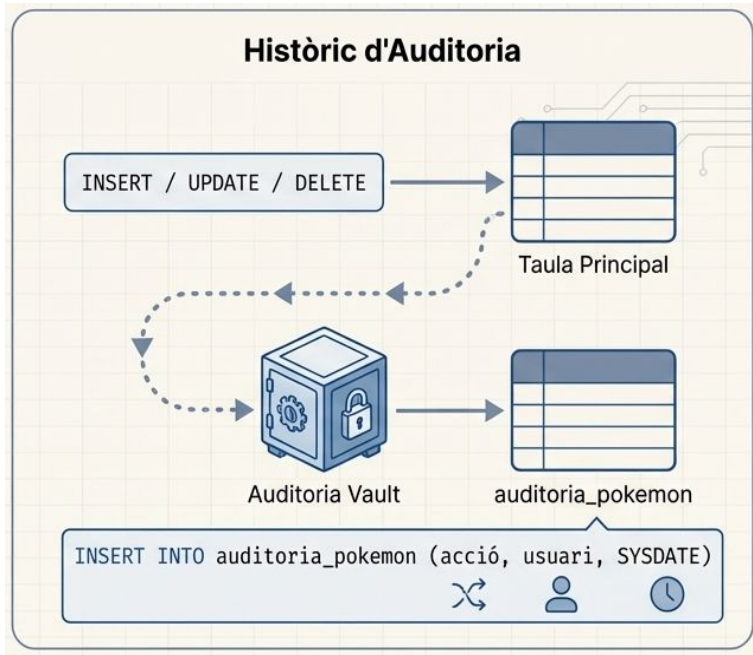
```
CREATE OR REPLACE TRIGGER evitar_eliminar
BEFORE DELETE ON empleats
FOR EACH ROW
DECLARE
PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
UPDATE empleats
SET esborrat = 'S', data_eliminacio = SYSDATE
WHERE id = :OLD.id;
COMMIT; -- Necessari en transacció autònoma

RAISE_APPLICATION_ERROR(-20002, 'Marcat com eliminat.');
```

```
END;
```

Triggers

- Triggers d'auditoria:
 - Serveixen per portar un registre de totes les operacions que es fan sobre alguna de les taules
 - Quí, Quan i qué normalment queda registrat



Triggers

- Per tant, caldrà crear una nova taula on deixar aquestes dades

```
CREATE TABLE auditar_empleats (  
    usuari varchar2(30),  
    accio varchar2(20),  
    data date  
);
```

Triggers

- Per tant, caldrà crear una nova taula on deixar aquestes dades

```
create or replace trigger tr_empl
before delete or update or insert on empleats
BEGIN
    if deleting THEN
        insert into auditar_empleats values(user, 'Borrat', sysdate);
    end if;
    if updating THEN
        insert into auditar_empleats values(user, 'update', sysdate);
    end if;
    if inserting THEN
        insert into auditar_empleats values(user, 'Insert', sysdate);
    end if;
END;
```